

HDH: A Python Library for Hypergraph-Based Distributed Quantum Computing Partitioning

Maria Gragera Garces ^{1*}

¹ University of Edinburgh * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Owen Lockwood](#) 

Reviewers:

- [@MIWdIB](#)
- [@oblivateandsurrender](#)

Submitted: 17 December 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

State of the Field

Quantum computing aims to solve computational problems that are classically hard. To achieve this in utility settings, quantum computers will require thousands if not millions of qubits. Current devices hold hundreds of qubits at most. It is believed that the path towards these scales will come from distribution, meaning the collaboration of various devices to complete tasks larger than their individual capacities. The main goal behind Distributed Quantum Computing (DQC) is to allocate sub-partitions of large quantum computations across multiple devices smaller than the computation itself. Existing approaches abstract computations to hypergraphs which are then partitioned, using BALANCED HYPERGRAPH PARTITIONING solvers such as KaHyPar ([Schlag et al., 2023](#)).

This framing has two fundamental limitations: \ 1) It reduces distribution to a balanced partitioning problem that ignores a hard physical constraint: individual QPUs have fixed qubit capacities, and a valid distribution must respect these limits strictly rather than treating them as soft penalties. \ 2) Existing hypergraph abstractions are model-specific and encode only a subset of possible partition cuts, meaning partitioning strategies are routinely evaluated on inconsistent abstractions, making cross-comparison unreliable and hindering the systematic development of improved heuristics.

The HDH library aims to tackle both of these problems, providing a unified and accessible starting point for the research of DQC partitioners.

Statement of need

HDH is a Python package designed for researchers to test and develop partitioning strategies for quantum workloads.

HDHs (Hybrid Dependency Hypergraphs) are an abstraction which transforms quantum computation, originating from any quantum computational model (including circuits, measurement-based quantum computing, quantum cellular automata, and quantum walks), to a directed hypergraph that expresses all possible partitions available within the computation. They were originally proposed in ([Gragera Garces et al., 2025](#)) as a unifying approach to quantum distribution, extending the hypergraph abstraction method for partitioning across devices originally proposed in ([Andrés-Martínez & Heunen, 2019](#)). Various partitioning strategies have since been proposed building on that earlier abstraction ([Clark et al., 2023](#); [Escofet et al., 2023](#); [Sundaram & Gupta, 2023](#)), but many are tested on inconsistent hypergraph abstractions, hindering cross-partitioner comparison and improvement.

Having an easy to implement, open-source, and model-agnostic abstraction will enable the fair and consistent cross-comparison of partitioning strategies in future work. Furthermore, HDHs extend this capability beyond the circuit model, addressing a current blind spot in DQC research.

41 HDH is designed to be used by both distributed quantum architecture researchers and compiler
42 developers. While libraries for distributed quantum computing exist (DISQCO (Burt et al.,
43 2026), Qdislib (Tejedor et al., 2025), Optyx (Kupper et al., 2025), DC-MBQC (Xue et al.,
44 2026)), these implement end-to-end distribution pipelines rather than exposing the underlying
45 abstraction as a research tool. No existing library provides a model-agnostic hypergraph
46 abstraction designed specifically to enable the development and fair comparison of partitioning
47 heuristics (the role HDH is built to fill).

48 While quantum compilation frameworks like Qiskit (Javadi-Abhari, A., Treinish, M., Krsulich,
49 K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P., Bishop, L. S., Cross, A.
50 W., Johnson, B. R., & Gambetta, J. M., 2024), Cirq (Developers, 2025), and PennyLane
51 (Bergholm et al., 2018) provide circuit optimization and device mapping, they do not offer
52 model-agnostic abstractions for distributed quantum computing. The HDH library is compatible
53 with these SDKs, making it a seamless addition to state-of-the-art quantum software stacks.

54 Software Design

55 The central design decision in HDH was to separate the abstraction layer from the partitioning
56 layer. Rather than building a monolithic tool that both constructs hypergraphs and partitions
57 them, HDH exposes the HDH as a first-class data structure that any downstream partitioner can
58 consume. This makes the library useful both as a standalone research tool and as a substrate
59 for third-party heuristics.

60 A hypergraph-based representation was chosen deliberately over simpler graph alternatives,
61 as quantum computing models frequently involve operations with more than two inputs or
62 outputs (a Toffoli gate, for instance, acts on three qubits simultaneously), requiring multi-way
63 correlations.

64 The library includes a capacity-aware greedy heuristic as a built-in baseline. Existing DQC
65 research typically benchmarks against KaHyPar (Schlag et al., 2023), a general-purpose
66 hypergraph partitioner not designed for quantum hardware constraints. While simpler than
67 KaHyPar, the included heuristic enforces per-device qubit capacity, making it a more appropriate
68 DQC baseline and a concrete starting point for researchers developing improved strategies.

69 Finally, HDH is implemented in Python to minimise the barrier to adoption. The quantum
70 software community has converged on Python as its primary language, and compatibility with
71 Qiskit, Cirq, and PennyLane was prioritised from the outset to ensure HDH integrates naturally
72 into existing workflows.

73 Model conversions

74 Any quantum computing model comprises a series of commands which establish qubit state
75 rotations, measurements and entanglements. For instance, quantum circuits are comprised
76 of a sequence of quantum gates applied to qubits. Single-qubit gates perform rotations on
77 the Bloch sphere, while multi-qubit gates (such as CNOT) create entanglement dependencies
78 between qubits.

79 HDHs use the following notation to describe quantum workload dependencies, including
80 predicted elements that represent potential future state transformations based on classical
81 measurement outcomes:

Symbol	Meaning
●	Classical node
•	Quantum node
==	Classical hyperedge
—	Quantum hyperedge
○	Predicted node
--	Predicted hyperedge

Figure 1: HDH symbol legend.

Mapping a quantum workload such as a circuit to an HDH involves applying specific correspondences between model elements and hypergraph motifs. This library provides model-specific classes such as the Circuit class that enable straightforward conversions to HDHs using mapping tables:

Motif	Operation
•	Qubit initialisation
•—•	Single qubit gate
•—•—•	Two qubit gate
•—•—•—•	Three qubit gate
—	Measurement

Figure 2: Circuit to HDH mapping table.

In the context of DQC, entangling operations in a model can be made non-local (namely non-local gates) and thus partitioned through a quantum network via quantum communication primitives (Anbang, W. and Hezi, Z. and Gushu, L and Alireza, S and Yuan, X and Yufei, D, 2022). Alternatively, qubit states can be individually forwarded through teleportation protocols (Zomorodi-Moghadam et al., 2017). HDHs aim to showcase all possible partitionings, thus enabling heuristic partitioners to exploit recurring patterns when mapping workloads to quantum or hybrid networks, thereby minimizing communication and other costs.

The table below shows how HDHs supersede previous abstractions in their expressivity of these partitioning options. Unlike prior approaches that represent only non-local gates or only teleportation, HDHs capture both strategies simultaneously, enabling partitioners to optimize across all available distribution methods:

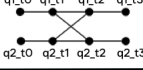
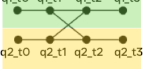
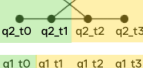
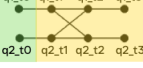
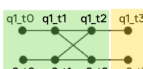
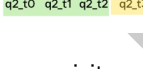
	Telegate	Teledata	HDH
			
Gate cut			
Wire cut			
			
			
			

Figure 3: Table showing HDH expressivity.

97 The library provides model-specific classes such as the Circuit class to enable workload to
98 HDH translation:

```
import hdh
from hdh.models.circuit import Circuit
from hdh.visualize import plot_hdh

circuit = Circuit()

# Set of instructions
circuit.add_instruction("ccx", [0, 1, 2])
circuit.add_instruction("h", [3])
circuit.add_instruction("h", [5])
circuit.add_instruction("cx", [3, 4])
circuit.add_instruction("cx", [2, 1])

circuit.add_conditional_gate(5, 4, "z")

circuit.add_instruction("cx", [0, 3])
circuit.add_instruction("measure", [2])
circuit.add_instruction("measure", [4])

hdh = circuit.build_hdh() # Generate HDH
fig = plot_hdh(hdh) # Visualize HDH
```

99 The resulting HDH is shown below as a graph representation of a hypergraph, since visualizing
100 large, multi-colored hypergraphs directly becomes impractical at scale. Gates have hyperedges
101 corresponding to the qubit state transformations they generate, as well as preceding and
102 following hyperedges that capture pre- and post-teleportation of the involved states. HDHs
103 differ from previous abstractions in two key ways:

- 104 (1) nodes represent possible state transformations rather than individual qubits or operations,
105 and
- 106 (2) classical data flows are explicitly included (shown in orange):

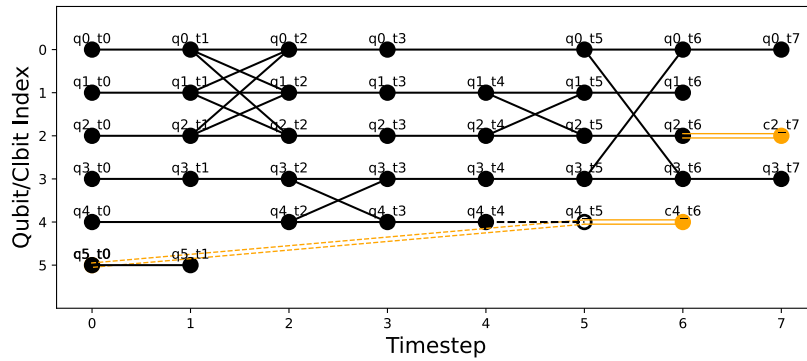


Figure 4: Example circuit and its HDH representation.

Research Impact Statement

HDH is the software implementation of a theoretical framework currently under peer review, as the library and its formal foundations were developed in tandem. The manuscript is available from the author upon request.

Early community engagement has been encouraging. The project was presented as a poster at SIGCOMM 2025 (Gragera Garces et al., 2025) (a major networking venue), has received funding through the Unitary Fund microgrant program (dedicated to supporting open source quantum software to benefit humanity) and has already seen external contributors (acknowledged below). Further, we're in discussion with companies in the Distributed Quantum Computing space regarding the library's integration within their stack.

The library is designed for immediate use by two audiences: distributed quantum architecture researchers who need a consistent abstraction to evaluate their partitioning strategies, and compiler developers building on existing framework who require a model-agnostic distribution layer. The open-source release and documentation are intended to lower the barrier to reproducible research in DQC partitioning.

Acknowledgements

We acknowledge contributions from Joseph Tedds, Manuel Alejandro, and Alessandro Cosentino.

We thank Unitary Fund for supporting this project through their quantum microgrant program.

The work of the author is supported by the EPSRC UK Quantum Technologies Programme under grant EP/T001062/1 and VeriQloud.

AI Usage Disclosure

Claude was used during both library development and paper writing.

In the library, Claude generated initial draft code implementations that were subsequently rewritten by the author. It also assisted in producing unit tests, which were validated against expected behaviour across both passing and failing scenarios. These were not always fully rewritten but they were revised and thoroughly tested. Additionally, AI was used to generate inline code comments throughout the library, with the aim of improving readability for contributors and users who may check the source code.

In the paper, Claude was used to assist with wording and polish.

136 All AI-generated content (both code and text) was reviewed or modified (as per the above
137 descriptions) by the author before inclusion in the present version.

138 References

- 139 Anbang, W. and Hezi, Z. and Gushu, L and Alireza, S and Yuan, X and Yufei, D. (2022).
140 AutoComm: A Framework for Enabling Efficient Communication in Distributed Quantum
141 Programs. *55th IEEE/ACM International Symposium on Microarchitecture*. [https://
142 ieexplore.ieee.org/abstract/document/9923799?casa_token=1tvAGu_y3NIAAAAAA:
143 26TLU9SEhr9YW01oTLUK9dQIVcH4OUQQ1DPdk1ZQNGU76kjlQmZ6xvycLJI9ltNldPbjCBafw](https://ieeexplore.ieee.org/abstract/document/9923799?casa_token=1tvAGu_y3NIAAAAAA:26TLU9SEhr9YW01oTLUK9dQIVcH4OUQQ1DPdk1ZQNGU76kjlQmZ6xvycLJI9ltNldPbjCBafw)
- 144 Andrés-Martínez, P., & Heunen, C. (2019). Automated distribution of quantum circuits via
145 hypergraph partitioning. *Physical Review A*. [https://journals.aps.org/pra/abstract/10.
146 1103/PhysRevA.100.032308](https://journals.aps.org/pra/abstract/10.1103/PhysRevA.100.032308)
- 147 Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M. S., Alonso-
148 Linaje, G., AkashNarayanan, B., Asadi, A., & others. (2018). PennyLane: Automatic
149 differentiation of hybrid quantum-classical computations. *arXiv Preprint arXiv:1811.04968*.
150 <https://arxiv.org/abs/1811.04968>
- 151 Burt, F., Chen, K.-C., & Leung, K. K. (2026). A multilevel framework for partitioning quantum
152 circuits. *Quantum*, 10, 1984.
- 153 Clark, J., Humble, T., & Thapliyal, H. (2023). TDAG: Tree-based Directed Acyclic Graph
154 Partitioning for Quantum Circuits. *Proceedings of the Great Lakes Symposium on VLSI*.
155 <https://dl.acm.org/doi/10.1145/3583781.3590234>
- 156 Developers, C. (2025). *Cirq*. Zenodo. <https://doi.org/10.5281/ZENODO.4062499>
- 157 Escofet, P., Ovide, A., Almudever, C. G., Alarcón, E., & Abadal, S. (2023). Hungarian Qubit
158 Assignment for Optimized Mapping of Quantum Circuits on Multi-Core Architectures.
159 *IEEE COMPUTER ARCHITECTURE LETTERS*. [https://ieeexplore.ieee.org/abstract/
160 document/10262036?casa_token=h_0XHyNKYYgAAAAA:CK2iz7lfvOP8pQAYzCkf0muy8e_
161 sYFz2Ql1AC517ngFIPgB0_4OEsnMY5JObDYgT-l3KQFIPA](https://ieeexplore.ieee.org/abstract/document/10262036?casa_token=h_0XHyNKYYgAAAAA:CK2iz7lfvOP8pQAYzCkf0muy8e_sYFz2Ql1AC517ngFIPgB0_4OEsnMY5JObDYgT-l3KQFIPA)
- 162 Gragera Garces, M., Heunen, C., & Marina, M. K. (2025). Distributed Quantum Computing
163 Across Heterogeneous Hardware with Hybrid Dependency Hypergraphs. *Proceedings
164 of the ACM SIGCOMM 2025 Posters and Demos*. [http://adsabs.harvard.edu/abs/
165 2017arXiv170304627P](http://adsabs.harvard.edu/abs/2017arXiv170304627P)
- 166 Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel,
167 S., Nation, P., Bishop, L. S., Cross, A. W., Johnson, B. R., & Gambetta, J. M. (2024).
168 Quantum computing with Qiskit. *ArXiv e-Prints*. arxiv.org/abs/2405.08810
- 169 Kupper, M., Yeung, R., Poór, B., Toumi, A., Cashman, W., & Felice, G. de. (2025).
170 Optyx: A ZX-based python library for networked quantum architectures. *arXiv Preprint
171 arXiv:2512.09648*.
- 172 Schlag, S., Heuer, T., Gottesbüren, L., Akhremtsev, Y., Schulz, C., & Sanders, P. (2023).
173 High-quality hypergraph partitioning. *ACM Journal of Experimental Algorithmics*, 27,
174 1–39.
- 175 Sundaram, R. G., & Gupta, H. (2023). Distributing Quantum Circuits Using Teleportations.
176 *IEEE International Conference on Quantum Software (QSW)*. [https://www.computer.org/
177 csdl/proceedings-article/qsw/2023/047900a186/1Q5oLz6DIZ2](https://www.computer.org/csdl/proceedings-article/qsw/2023/047900a186/1Q5oLz6DIZ2)
- 178 Tejedor, M., Casas, B., Conejero, J., Cervera-Lierta, A., & Badia, R. M. (2025). Orchestrating
179 quantum-HPC workflows with distributed quantum circuit cutting. *Proceedings of the SC'25
180 Workshops of the International Conference for High Performance Computing, Networking,
181 Storage and Analysis*, 1898–1906.

- 182 Xue, Y., Yang, R., Liang, Z., & Li, T. (2026). DC-MBQC: A distributed compilation framework
183 for measurement-based quantum computing. *arXiv Preprint arXiv:2601.00214*.
- 184 Zomorodi-Moghadam, M., Houshmand, M., & Houshmand, M. (2017). Optimizing
185 Teleportation Cost in Distributed Quantum Circuits. *International Journal of Theoretical*
186 *Physics*. <https://link.springer.com/article/10.1007/s10773-017-3618-x>

DRAFT